Deep learning for Bitcoin transaction and address classification: week 1

Oct 30, 2026 · @Ren

Summary

On both datasets tested so far, tree models beat the neural networks: Random Forest reaches an illicit F1 of 0.826 on Elliptic against 0.678 for the best graph neural network, and XGBoost reaches a 10-class macro F1 of 0.832 on BABD-13 against 0.732 for an MLP.

- **Transactions (Elliptic, illicit vs licit):** a plain MLP, a regularized MLP, a GCN and a GraphSAGE were trained with PyTorch and PyTorch Geometric and compared with a Random Forest on the standard temporal split.
- **Addresses (BABD-13, 13 behaviour classes):** an MLP was compared with XGBoost; XGBoost reproduces the published accuracy (0.970 against 0.967).
- **The graph helps the neural networks:** with the same features, GCN and GraphSAGE raise illicit F1 by 0.07 to 0.11 over the MLP, mostly by cutting false alarms from 389 to about 150.
- **No model survives the drift on Elliptic:** after the dark-market shutdown at time step 43, every model catches at most 1.8% of illicit transactions.

Objective and datasets

The professor's directive is to try deep learning to classify Bitcoin transactions and addresses, and to relate it to the literature. Week 1 covers one transaction dataset and one address dataset, each with a classical baseline and at least one neural model.

Dataset	Unit	Size used	Classes	Source
Elliptic (Weber et al. 2019)	transaction	203,769 nodes, 234,355 edges, 46,564 labelled	illicit vs licit	Kaggle
BABD-13 (Xiang et al. 2022)	address	544,462 addresses, 148 features	13 behaviour classes	Kaggle (`BABD-13.csv` only)

Real-CATS was planned as the address dataset, but the Kaggle copy downloaded holds only Ethereum transaction records (77.6 GB uncompressed) and no Bitcoin address profiles, so BABD-13 took its place. The BABD archive also contains per-address subgraph

files (22.4 GB), not used yet.

Method

Every neural model was compared with a tree-based baseline on the same split and features, and judged on the rare class, not on accuracy.

- **Elliptic split:** temporal, as in Weber et al. Training on time steps 1–34 (29,894 labelled transactions, 11.6% illicit), test on steps 35–49 (16,670, 6.5% illicit). Features standardized with statistics from the training period only.
- **BABD-13 split:** stratified random 80/20 (435,569 / 108,893 addresses), as in the paper; the file has no time or block column, so no temporal split is possible. The `SW` column (strong or weak label) was excluded from the features because it describes where a label came from, not how the address behaves.
- **Imbalance:** class weights in every model (`pos_weight` in the binary loss; square-root balanced weights in the 13-class loss, since full balancing would weight a Ponzi error about 2,800 times an exchange error).
- **Metrics:** precision, recall and F1 on the illicit class plus PR AUC (Elliptic); accuracy, macro F1 over 13 classes, macro F1 over 10 classes (without the three classes of 27 addresses or fewer), and weighted F1 (BABD-13).
- **Leakage checks:** no Elliptic edge crosses time steps (checked: 0 of 234,355), so message passing never brings test-period information into training nodes.
- **Environment:** Python 3.14, PyTorch 2.15 (CUDA 13.2), PyTorch Geometric 2.8.1, scikit-learn, XGBoost; NVIDIA RTX 3050 6 GB laptop GPU. Each neural result comes from one run with seed 42.

Elliptic: transaction classification

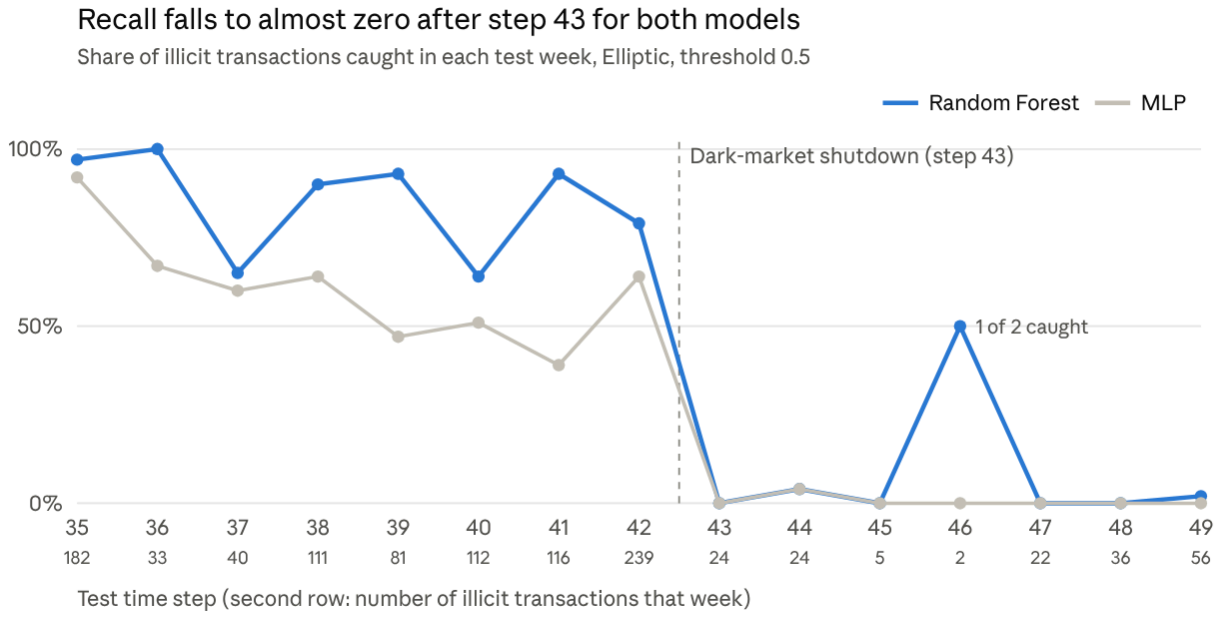
The Random Forest is best by a wide margin (illicit F1 0.826); among neural models, the graph networks beat the MLP by 0.07 to 0.11 F1.

Model	Precision	Recall	F1	PR AUC	False alarms
Random Forest (200 trees)	0.963	0.723	**0.826**	**0.793**	30
GraphSAGE (2 layers, 64 units)	0.806	0.584	0.678	0.640	152
GCN (2 layers, 64 units)	0.788	0.528	0.632	0.649	154
MLP (165-64-32-1)	0.599	0.536	0.565	0.528	389
MLP + validation, early stopping, dropout	0.421	0.513	0.463	0.291	764

Test set: 16,670 transactions, 1,083 illicit; threshold 0.5 except the regularized MLP (0.534, chosen on validation).

- **The graph mainly improves precision.** On identical features, adding the links cuts false alarms from 389 (MLP) to about 150; recall barely moves.
- **The GCN reproduces the paper:** F1 0.632 against 0.628 in Weber et al.
- **Regularizing the MLP made it worse.** Validation on steps 30–34 rose to PR AUC 0.864 while test PR AUC fell to 0.291: the recent past did not predict the later test period, and early stopping never triggered (best epoch 490 of 500).
- *Hypothesis, not tested:* 72 of Elliptic's 165 features already aggregate each transaction's neighbours, so the Random Forest gets much of the graph information through its inputs.

Drift after step 43. Recall on steps 35–42 against 43–49: Random Forest 0.853 against 0.018, GraphSAGE 0.690 against 0.012, MLP 0.633 against 0.006, GCN 0.625 against 0.006. All 169 illicit transactions after the dark-market shutdown are almost entirely missed, by every model, as Weber et al. also report.



Notebook output, test time steps 35–49 · Random Forest and plain MLP

Before step 43 the Random Forest stays above the MLP almost every week; after it, both lines sit at or near zero, so the drift is not a weakness of one model.

BABD-13: address classification

XGBoost reproduces the paper (accuracy 0.970 against 0.967) and beats the MLP on every summary metric.

Model	Accuracy	Macro F1 (13 classes)	Macro F1 (10 classes)	Weighted F1
XGBoost (300 trees, depth 8)	**0.970**	**0.662**	**0.832**	**0.969**
MLP (148-256-128-13, dropout 0.2)	0.921	0.604	0.732	0.922

Test set: 108,893 addresses. The MLP used a sign-log transform of the features, mini-batches of 1,024 and the best of 30 epochs on a 10% validation set.

- **Eight classes are well recognized** (F1 0.86 to 0.99 for XGBoost, 0.65 to 0.96 for the MLP), including darknet market (XGBoost 0.992) and tumbler (0.858).
- **The three tiny classes are noise.** Government blacklist, money laundering and Ponzi have 5, 3 and 3 test addresses. XGBoost scores 0.00, 0.29 and 0.00 on them; the MLP, pushed by its class weights, finds more of them but with precision 0.04 to 0.33.
- **Individual wallet and blackmail are hard for both** (XGBoost F1 0.07 and 0.59). *Hypothesis, not tested:* according to the BABD paper these classes rely mostly on weak labels from user reports, so their labels may be unreliable; the `SW` column would allow checking this.
- **The MLP's validation score was unstable** (macro F1 between 0.52 and 0.68 from one epoch to the next), because the validation set holds only one or two addresses of each tiny class; the 10-class macro F1 would be a steadier selection criterion.

Findings and limitations

The week-1 answer to "does deep learning help?" is: not on these features, though the graph narrows the gap on Elliptic.

1. **Trees beat neural networks on tabular features, on both datasets.** The gap is 0.15 F1 on Elliptic (Random Forest against GraphSAGE) and 0.10 macro F1 on BABD-13 (XGBoost against MLP).
2. **The graph structure carries information.** GNNs beat the MLP on Elliptic with identical node features, mainly through precision.
3. **Drift dominates Elliptic.** No model, classical or deep, detects the illicit transactions after the step-43 shutdown; validating on the recent past did not reveal this.
4. **Label quality limits BABD-13.** The weakest classes are tiny (3 to 5 test addresses) or, according to the paper, mostly weakly labelled.

Limitations

- Neural results come from one run and one seed each; another seed could move them by a few points, though not enough to change the ranking.
- Hyperparameters of the neural models were barely tuned (one architecture each, fixed learning rate); the tree baselines were not tuned either.
- BABD-13 uses a random split: its scores say nothing about drift, unlike Elliptic.
- Nothing here touches the dark-web addresses of the Venom dataset yet.

Next steps

The next experiment is a graph model on BABD-13's per-address subgraphs, the one setting where the paper ran no deep graph model, followed by the Venom application; the plan ends on `Nov 10, 2026`.

- [ ] Open a few BABD subgraph files and document their format, then extract the archive (22.4 GB; 50 GB free).
- [ ] Train a graph-level model (one subgraph per address) and compare it with XGBoost on the same split.
- [ ] Repeat the main neural runs with three seeds to give a spread instead of one number.
- [ ] After the subgraph model: apply a trained model to the eight Venom address sets, if the features can be reproduced from the downloaded transactions.
- [ ] Ask the professor whether the Ethereum part of Real-CATS (already downloaded) is of interest.

Appendix: code

The cells below ran in order in one notebook (`3_deep_learning.ipynb`); outputs are in the sections above.

A.1 Elliptic: data and temporal split

```
import pandas as pd, numpy as np, torch
from sklearn.preprocessing import StandardScaler

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
DATA = "../elliptic"
feats = pd.read_csv(f"{DATA}/elliptic_txs_features.csv", header=None)
classes = pd.read_csv(f"{DATA}/elliptic_txs_classes.csv")
feats.columns = ["txId", "time_step"] + [f"f{i}" for i in range(165)]

d = feats.merge(classes, on="txId")
d = d[d["class"] != "unknown"].copy()
d["y"] = (d["class"] == "1").astype(int)
```

```
tr, te = d[d["time_step"] <= 34], d[d["time_step"] > 34]
xcols = [c for c in d.columns if c.startswith("f")]

scaler = StandardScaler().fit(tr[xcols])
Xtr_t = torch.tensor(scaler.transform(tr[xcols]), dtype=torch.float32,
device=device)
Xte_t = torch.tensor(scaler.transform(te[xcols]), dtype=torch.float32,
device=device)
ytr_t = torch.tensor(tr["y"].values, dtype=torch.float32, device=device)
yte_t = torch.tensor(te["y"].values, dtype=torch.float32, device=device)
```

A.2 Elliptic: MLP and Random Forest

```
import torch.nn as nn
from sklearn.metrics import classification_report, average_precision_score,
confusion_matrix
from sklearn.ensemble import RandomForestClassifier

torch.manual_seed(42)
model = nn.Sequential(nn.Linear(165, 64), nn.ReLU(), nn.Linear(64, 32),
nn.ReLU(), nn.Linear(32, 1)).to(device)
pos_weight = ((ytr_t == 0).sum() / (ytr_t == 1).sum()).reshape(1)
loss_fn = nn.BCEWithLogitsLoss(pos_weight=pos_weight)
opt = torch.optim.Adam(model.parameters(), lr=1e-3)
for epoch in range(300):
    model.train(); opt.zero_grad()
    loss = loss_fn(model(Xtr_t).squeeze(1), ytr_t); loss.backward();
opt.step()

model.eval()
with torch.no_grad():
    proba = torch.sigmoid(model(Xte_t).squeeze(1)).cpu().numpy()
y_te = yte_t.cpu().numpy().astype(int)
pred = (proba >= 0.5).astype(int)

rf = RandomForestClassifier(n_estimators=200, n_jobs=-1, random_state=42)
rf.fit(Xtr_t.cpu().numpy(), tr["y"].values)
rf_proba = rf.predict_proba(Xte_t.cpu().numpy())[:, 1]
rf_pred = (rf_proba >= 0.5).astype(int)
```

A.3 Elliptic: regularized MLP (validation on steps 30–34, early stopping, dropout, weight decay)

```
import copy
from sklearn.metrics import precision_recall_curve

fit_mask = torch.tensor((tr["time_step"] <= 29).values, device=device)
Xf, yf, Xv, yv = Xtr_t[fit_mask], ytr_t[fit_mask], Xtr_t[~fit_mask],
ytr_t[~fit_mask]
yv_np = yv.cpu().numpy().astype(int)
torch.manual_seed(42)
model = nn.Sequential(nn.Linear(165, 64), nn.ReLU(), nn.Dropout(0.2),
                      nn.Linear(64, 32), nn.ReLU(), nn.Dropout(0.2),
nn.Linear(32, 1)).to(device)
loss_fn = nn.BCEWithLogitsLoss(pos_weight=((yf == 0).sum() / (yf ==
1).sum()).reshape(1))
opt = torch.optim.Adam(model.parameters(), lr=1e-3, weight_decay=1e-4)
best_ap, best_state = -1, None
for epoch in range(500):
    model.train(); opt.zero_grad()
    loss = loss_fn(model(Xf).squeeze(1), yf); loss.backward(); opt.step()
    if epoch % 10 == 0:
        model.eval()
        with torch.no_grad():
            pv = torch.sigmoid(model(Xv).squeeze(1)).cpu().numpy()
        ap = average_precision_score(yv_np, pv)
        if ap > best_ap:
            best_ap, best_state = ap, copy.deepcopy(model.state_dict())
model.load_state_dict(best_state); model.eval()
with torch.no_grad():
    pv = torch.sigmoid(model(Xv).squeeze(1)).cpu().numpy()
    proba2 = torch.sigmoid(model(Xte_t).squeeze(1)).cpu().numpy()
p, r, t = precision_recall_curve(yv_np, pv)
thr = t[np.argmax((2 * p * r / (p + r + 1e-12))[:-1])]
pred2 = (proba2 >= thr).astype(int)
```

A.4 Elliptic: graph and GNNs (PyTorch Geometric)

```
from torch_geometric.data import Data
from torch_geometric.nn import GCNConv, SAGEConv
import torch.nn.functional as F

edges = pd.read_csv(f"{DATA}/elliptic_txs_edgelist.csv")
all_ = feats.merge(classes, on="txId", how="left")
idx = pd.Series(np.arange(len(all_)), index=all_["txId"])
src, dst = idx[edges["txId1"]].values, idx[edges["txId2"]].values
edge_index = torch.tensor(np.vstack([np.r_[src, dst], np.r_[dst, src]]),
dtype=torch.long)
lab = all_["class"].map({"1": 1, "2": 0}).fillna(-1).astype(int).values
```

```
step = all_["time_step"].values
scaler_g = StandardScaler().fit(all_.loc[step <= 34, xcols])
x = torch.tensor(scaler_g.transform(all_[xcols]), dtype=torch.float32)
train_mask = torch.tensor((lab >= 0) & (step <= 34)).to(device)
test_mask = torch.tensor((lab >= 0) & (step > 34)).to(device)
data = Data(x=x, edge_index=edge_index, y=torch.tensor(lab)).to(device)
print("edges crossing time steps:", int((step[src] != step[dst]).sum()))   #
0

class GNN(nn.Module):
    def __init__(self, conv, in_dim=165, hid=64, p=0.2):
        super().__init__()
        self.c1, self.c2, self.out, self.p = conv(in_dim, hid), conv(hid,
hid), nn.Linear(hid, 1), p
    def forward(self, x, ei):
        h = F.dropout(F.relu(self.c1(x, ei)), self.p, self.training)
        h = F.dropout(F.relu(self.c2(h, ei)), self.p, self.training)
        return self.out(h).squeeze(1)

y_f = data.y.float()
pw = ((data.y[train_mask] == 0).sum() / (data.y[train_mask] ==
1).sum()).float().reshape(1)

def run(conv, epochs=300, lr=5e-3):
    torch.manual_seed(42)
    m = GNN(conv).to(device)
    opt = torch.optim.Adam(m.parameters(), lr=lr, weight_decay=5e-4)
    lf = nn.BCEWithLogitsLoss(pos_weight=pw)
    for ep in range(epochs):
        m.train(); opt.zero_grad()
        loss = lf(m(data.x, data.edge_index)[train_mask], y_f[train_mask]);
loss.backward(); opt.step()
    m.eval()
    with torch.no_grad():
        sc = torch.sigmoid(m(data.x, data.edge_index))
[test_mask].cpu().numpy()
    return m, sc, (sc >= 0.5).astype(int)

gcn, gcn_sc, gcn_pred = run(GCNConv)
sage, sage_sc, sage_pred = run(SAGEConv)
```

A.5 BABD-13: data and XGBoost

```
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, f1_score
from xgboost import XGBClassifier

babd = pd.read_csv(r"C:\data\babd\BABD-13.csv")
feat_cols = [c for c in babd.columns if c not in ("account", "SW", "label")]
# 148 features
Xb, yb = babd[feat_cols].values.astype("float32"), babd["label"].values
Xb_tr, Xb_te, yb_tr, yb_te = train_test_split(Xb, yb, test_size=0.2,
stratify=yb, random_state=42)
xgb = XGBClassifier(n_estimators=300, max_depth=8, learning_rate=0.1,
tree_method="hist", device="cuda", random_state=42)
xgb.fit(Xb_tr, yb_tr)
xp = xgb.predict(Xb_te)
```

A.6 BABD-13: MLP

```
from torch.utils.data import TensorDataset, DataLoader

def slog(a):
    return np.sign(a) * np.log1p(np.abs(a))

Xb_fit, Xb_val, yb_fit, yb_val = train_test_split(Xb_tr, yb_tr,
test_size=0.1, stratify=yb_tr, random_state=42)
sc_b = StandardScaler().fit(slog(Xb_fit))
to_t = lambda a: torch.tensor(sc_b.transform(slog(a)), dtype=torch.float32)
Xf_b, Xv_b, Xt_b = to_t(Xb_fit), to_t(Xb_val), to_t(Xb_te)
loader = DataLoader(TensorDataset(Xf_b, torch.tensor(yb_fit)),
batch_size=1024, shuffle=True)
counts = np.bincount(yb_fit, minlength=13)
w = (counts.sum() / (13 * counts)) ** 0.5
w = torch.tensor(w / w.mean(), dtype=torch.float32, device=device)

torch.manual_seed(42)
mlp_b = nn.Sequential(nn.Linear(148, 256), nn.ReLU(), nn.Dropout(0.2),
                      nn.Linear(256, 128), nn.ReLU(), nn.Dropout(0.2),
nn.Linear(128, 13)).to(device)
opt = torch.optim.Adam(mlp_b.parameters(), lr=1e-3)
lf = nn.CrossEntropyLoss(weight=w)

def predict(m, X):
    m.eval(); out = []
    with torch.no_grad():
        for i in range(0, len(X), 8192):
            out.append(m(X[i:i + 8192].to(device)).argmax(1).cpu())
    return torch.cat(out).numpy()
```

```
best, best_state = -1, None
for epoch in range(30):
    mlp_b.train()
    for xb_, yb_ in loader:
        xb_, yb_ = xb_.to(device), yb_.to(device)
        opt.zero_grad(); loss = lf(mlp_b(xb_), yb_); loss.backward();
opt.step()
    vf = f1_score(yb_val, predict(mlp_b, Xv_b), average="macro")
    if vf > best:
        best, best_state = vf, copy.deepcopy(mlp_b.state_dict())
mlp_b.load_state_dict(best_state)
mp = predict(mlp_b, Xt_b)
```